Questions

Q1: 为何用2个线程计算pi的值反而不准?

	n			
	10^5	10^6	10^7	10^8
π	3.14159	3.141593	3.1415927	3.14159265
1 Thread	3.14158	3.141592	3.1415926	3.14159264
2 Threads	3.14158	3.141480	3.1413692	3.14164686

- **竞争条件（Race Condition）的问题，合并各线程的结果时出现了问题**
  - 如果两个线程共享了同一个变量，而多个线程同时对这个变量进行写操作，导致竞争条件

Q2: 线程之间是如何实现数据传递的?

- **线程通过访问共享变量或数据结构（如全局变量、堆内存、队列等）传递数据**

Q3: Busy-waiting与Mutex在实现多线程对临界区访问时有什么区别？哪个效率更高？

- **Busy-waiting：低延迟但高CPU占用，适合极短操作。**
  - 避免了内核态切换的开销
  - 内核态切换是指CPU在执行程序时，从用户态切换到内核态（或反之）的过程

- **Mutex：高延迟但节省CPU，适合通用场景。**

Q4: Mutex与Semaphore有什么区别？简单说一下。

特性	Mutex（互斥锁）	Semaphore（信号量）
用途	保护共享资源的**独占访问**（临界区）	控制**资源数量**或线程/进程的**同步**
持有者	必须由**加锁的线程解锁**（所有权概念）	可由**任意线程释放**（无所有权）
计数	只能是 **0（未锁）或 1（已锁）**	可以是 **任意非负整数**（允许多个访问）
线程阻塞	竞争失败的线程会**阻塞**	如果计数≤0，线程会**阻塞**
典型场景	保护全局变量、数据结构	限流（如数据库连接池）、生产者-消费者